

## Code-Layout, Readability and Reusability

|                      |               |
|----------------------|---------------|
| <b>Date</b>          | 15-07-2026    |
| <b>Team ID</b>       |               |
| <b>Project Name</b>  | Rising Waters |
| <b>Maximum Marks</b> | 5 Marks       |

### Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. The Rising Waters – Flood Prediction System Using Machine Learning project follows good coding standards by maintaining a well-organized folder structure, descriptive naming conventions, modular programming practices, and proper exception handling. These practices improve maintainability and simplify future enhancements.

### Code Layout Checklist

| S.No | Code Quality Parameter    | Description                                   | Followed (Yes / No / Partial) | Remarks                                                                                           |
|------|---------------------------|-----------------------------------------------|-------------------------------|---------------------------------------------------------------------------------------------------|
| 1    | Consistent Indentation    | Uniform spacing/tabs used throughout the code | Yes                           | Standard Python indentation (4 spaces) used consistently                                          |
| 2    | Proper File Structure     | Files and folders are logically organized     | Yes                           | Project separated into templates, static, dataset, and source code folders                        |
| 3    | Meaningful Variable Names | Variables reflect their purpose clearly       | Yes                           | Variables like rainfall, latitude, prediction, and total_deaths improve readability               |
| 4    | Function / Method Names   | Functions are descriptively named             | Yes                           | Functions such as home(), predict(), and model training methods are clearly named                 |
| 5    | Code Comments             | Inline and block comments explain logic       | Yes                           | Important sections such as dataset loading, preprocessing, training, and prediction are commented |
| 6    | Modular Design            | Code is split into reusable functions/modules | Yes                           | Separate files for model training (train_model.py) and application (app.py)                       |
| 7    | No Redundant Code         | Duplicate or unused code is removed           | Yes                           | Code optimized by removing unnecessary statements and duplicate logic                             |
| 8    | Error Handling            | Exceptions and errors are handled gracefully  | Yes                           | Flask application uses try-except blocks to manage runtime errors                                 |

### Reusable Components / Modules

| S.No | Component / Module Name                  | Language / Technology | Where Reused                       | Reusability Level |
|------|------------------------------------------|-----------------------|------------------------------------|-------------------|
| 1    | train_model.py                           | Python, Scikit-learn  | Model training and retraining      | High              |
| 2    | app.py                                   | Python, Flask         | Prediction API and web application | High              |
| 3    | HTML Templates (index.html)              | HTML5, Jinja2         | User Interface                     | High              |
| 4    | CSS Styles (style.css)                   | CSS3                  | Dashboard design and UI            | Medium            |
| 5    | Machine Learning Model (flood_model.pkl) | Joblib                | Prediction module                  | High              |

## Overall Code Quality Assessment

| Aspect                   | Rating (1–5) | Comments                                                                                                       |
|--------------------------|--------------|----------------------------------------------------------------------------------------------------------------|
| Code Layout & Structure  | 5            | Well-organized directory structure with separate modules                                                       |
| Readability              | 5            | Meaningful variable names, proper formatting, and consistent coding style                                      |
| Reusability              | 5            | Independent modules allow reuse in future projects                                                             |
| Documentation / Comments | 5            | Code sections are documented with descriptive comments                                                         |
| <b>Overall Score</b>     | <b>5 / 5</b> | <b>The project follows good software engineering practices with maintainable, readable, and reusable code.</b> |